

RAG 第二阶段：Query Embedding 与 Vector Retrieval

现在开始开发 RAG 项目的第二阶段。

严格基于第一阶段已经完成的代码继续开发。

不要重写第一阶段。

不要实现第三阶段 Gemini 最终回答。

本阶段只实现：

用户问题 → Query Embedding → Vectorize Search → Top-K → 返回相关 Chunk

一、本阶段目标

完成 RAG 的 Retrieval 部分。

用户输入：

“这个项目的主要功能是什么？”

系统执行：

1. 接收 Query。
2. 调用 Gemini Embedding。
3. 得到 Query Vector。
4. 调用 Cloudflare Vectorize。
5. 进行相似度搜索。
6. 获取 Top-K。
7. 返回相关 Chunk。
8. 前端展示检索结果。

本阶段不调用 Gemini LLM 生成最终回答。

二、后端 API

实现：

POST /api/search

请求：

```
{ "query": "...", "topK": 5 }
```

返回：

- query
- results
- score
- documentId
- filename
- chunkIndex
- text

具体 JSON 结构可以根据现有代码合理设计。

三、Query Embedding

复用第一阶段：

services/embedding.js

不要重新实现一套 Embedding。

确保：

Document Embedding

和

Query Embedding

使用同一个 Gemini Embedding 模型。

四、Vector Search

实现独立：

rag/retriever.js

职责：

query → embedding → Vectorize query → Top-K results

不要把 Retrieval 逻辑直接写进 HTTP Route。

Route 只负责：

参数验证 → 调用 retriever → 返回结果

五、Top-K

默认：

topK = 5

前端允许用户调整。

建议合理范围：

1 ~ 10

防止用户输入极端值。

六、检索结果

返回：

- similarity score
- filename
- documentId
- chunkIndex
- text

这样前端能够明确展示：

“这次检索到了哪些知识。”

不要在本阶段进行 LLM 总结。

七、前端 UI

右侧 AI 区域开始启用。

底部输入框可以输入问题。

点击发送：

调用：

POST /api/search

上方展示：

“检索结果”

每个结果显示：

- 文件名
- Chunk
- 相似度
- 文本内容

暂时不要生成 AI 回答。

可以在界面上明确区分：

Query

和

Retrieved Context。

这样方便调试 RAG。

八、RAG 参数

左侧继续保留：

Top K

默认 5。

Chunk Size / Chunk Overlap 继续展示。

但是注意：

修改 Chunk Size 和 Chunk Overlap：

只对后续重新上传 / 重新索引的文件生效。

不要误以为修改 UI 参数就可以自动改变已经存在的 Vector。

九、调试能力

开发阶段建议保留检索调试信息：

- Query
- Query Embedding 是否成功
- 返回数量
- Top-K
- similarity score
- filename
- chunkIndex

这些信息可以通过开发模式展示。

不要把 embedding 数组完整显示到用户界面。

十、测试

完成后测试：

1. 输入一个明确存在于文档中的问题。
2. 查看 Top-K。
3. 检查最相关 Chunk 是否排名靠前。
4. 输入无关问题。
5. 查看是否返回低相关度结果。
6. 调整 Top-K。
7. 测试不同文件之间的检索。
8. 删除文件后再次检索，确认已经删除的文件不会继续出现。

本阶段不实现：

- Gemini 最终回答
- Prompt
- Chat history
- Streaming

开发完成后停止。

请告诉我：

1. 修改哪些文件。
2. Retrieval 的完整代码路径。
3. Query Embedding 如何完成。
4. Vectorize Search 如何完成。
5. Top-K 如何实现。
6. 如何测试。
7. 如何判断 RAG Retrieval 是否有效。
8. 当前还有哪些限制。

不要自动进入第三阶段。